

```
===== FILE: __init__.py =====
```

```
__version__ = "0.3.29"
```

```
===== FILE: __main__.py =====
```

```
from .cli import main
```

```
if __name__ == "__main__":  
    raise SystemExit(main())
```

```
===== FILE: _compat.py =====
```

```
from __future__ import annotations
```

```
import shlex
```

```
from pathlib import Path
```

```
from typing import Iterable
```

```
def shlex_join(split_command: Iterable[str]) -> str:  
    """Backport of shlex.join (added in Python 3.8)."""  
    return " ".join(shlex.quote(arg) for arg in split_command)
```

```
def unlink_missing(path: Path) -> None:  
    """Backport of Path.unlink(missing_ok=True); the kwarg was added in Python 3.8."""  
    try:  
        path.unlink()  
    except FileNotFoundError:  
        pass
```

```
def remove_prefix(value: str, prefix: str) -> str:  
    """Backport of str.removeprefix (added in Python 3.9)."""  
    if prefix and value.startswith(prefix):  
        return value[len(prefix):]  
    return value
```

```
def remove_suffix(value: str, suffix: str) -> str:  
    """Backport of str.removesuffix (added in Python 3.9)."""
```

```
    if suffix and value.endswith(suffix):  
        return value[: -len(suffix)]  
  
    return value
```

===== FILE: cli.py =====

```
from __future__ import annotations  
  
import argparse  
  
import hashlib  
  
import json  
  
import os  
  
import select  
  
import re  
  
import secrets  
  
import signal  
  
import shutil  
  
import subprocess  
  
import sys  
  
import threading  
  
import time  
  
import urllib.parse  
  
import urllib.request  
  
import uuid  
  
from collections.abc import Callable, Sequence  
  
from pathlib import Path  
  
from socket import gethostname  
  
from . import __version__  
  
from ._compat import remove_prefix, shlex_join, unlink_missing  
  
from .cloud import DEFAULT_CLOUD_URL, CloudClient, CloudConfig, CloudSyncer, InterruptWatche...  
  
from .crypto import decrypt_account_key, generate_pair_keypair  
  
from .server import serve  
  
from .store import RunStore, default_db_path  
  
RESERVED_COMMANDS = {  
    "run",  
    "server",  
    "status",  
    "public",  
    "ngrok",
```

```

    "login",

    "heartbeat",

    "cloud-login",

    "cloud-logout",

    "cloud-status",

    "project",

    "doctor",

    "sync",

    "version",

    "update",

}

PUBLIC_URL_RE = re.compile(r"https://[a-zA-Z0-9.-]+\.trycloudflare\.com")

HEARTBEAT_INTERVAL_SECONDS = 60

HEARTBEAT_ACTIVE_POLL_SECONDS = 3

ORPHANED_RUN_GRACE_SECONDS = 30

INTERRUPT_NOTE = "\n[??] Interrupted from mobile app.\n"


def main(argv: Sequence[str] | None = None) -> int:

    args = list(sys.argv[1:] if argv is None else argv)

    if not args:

        print_help()

        return 2

    first = args[0]

    if first in {"-h", "--help"}:

        print_help()

        return 0

    if first == "server":

        return server_command(args[1:])

    if first == "public":

        return public_command(args[1:])

    if first == "ngrok":

        return ngrok_command(args[1:])

    if first == "status":

        return status_command(args[1:])

    if first == "login":

        return pairing_login_command(args[1:])

    if first == "heartbeat":

        return heartbeat_command(args[1:])

    if first == "cloud-login":

        return cloud_login_command(args[1:])

```

```

if first == "cloud-logout":

    return cloud_logout_command(args[1:])

if first == "cloud-status":

    return cloud_status_command(args[1:])

if first == "project":

    return project_command(args[1:])

if first == "doctor":

    return doctor_command(args[1:])

if first == "sync":

    return sync_command(args[1:])

if first == "version":

    return version_command(args[1:])

if first == "update":

    return update_command(args[1:])

if first == "run":

    command, project_override = parse_run_args(args[1:])

    return run_command(command, project_override=project_override)

command, project_override = parse_run_args(args)

return run_command(command, project_override=project_override)

```

```

def server_command(argv: Sequence[str]) -> int:

    parser = argparse.ArgumentParser(prog="hao server")

    parser.add_argument("--host", default="127.0.0.1")

    parser.add_argument("--port", type=int, default=8765)

    parser.add_argument(

        "--token",

        default=None,

        help="Bearer token required by /api/* endpoints. Can also use HAOLEME_TOKEN.",

    )

    ns = parser.parse_args(argv)

    serve(ns.host, ns.port, token=ns.token)

    return 0

```

```

def public_command(argv: Sequence[str]) -> int:

    parser = argparse.ArgumentParser(prog="hao public")

    parser.add_argument("--port", type=int, default=8765)

    parser.add_argument("--host", default="127.0.0.1")

    parser.add_argument("--token", default=None)

    parser.add_argument("--cloudflared", default="cloudflared")

```

```

ns = parser.parse_args(argv)

cloudflared = shutil.which(ns.cloudflared) if ns.cloudflared == "cloudflared" else ns.cl...

if not cloudflared:

    print(

        "hao: cloudflared is required for public tunnels.\n"

        "Install it with: brew install cloudflared\n"

        "Linux: https://developers.cloudflare.com/cloudflare-one/connections/connect-net...

        file=sys.stderr,

    )

    return 127

token = ns.token or os.environ.get("HAOLEME_TOKEN") or secrets.token_urlsafe(24)

local_url = f"http://{ns.host}:{ns.port}"

print("Starting ??? local server...")

print(f"Local: {local_url}")

print(f"Token: {token}")

print("Starting Cloudflare quick tunnel...")

server_thread = threading.Thread(

    target=serve,

    args=(ns.host, ns.port),

    kwargs={"token": token},

    daemon=True,

)

server_thread.start()

time.sleep(0.5)

proc = subprocess.Popen(

    [cloudflared, "tunnel", "--url", local_url],

    stdout=subprocess.PIPE,

    stderr=subprocess.STDOUT,

    text=True,

    bufsize=1,

)

public_url_seen = threading.Event()

def stream_tunnel_output() -> None:

    if proc.stdout is None:

        return

    for line in proc.stdout:

        sys.stdout.write(line)

```

```

        sys.stdout.flush()

        match = PUBLIC_URL_RE.search(line)

        if match and not public_url_seen.is_set():
            public_url_seen.set()

            print()

            print("Use this in the Android app:")

            print(f"Server: {match.group(0)}")

            print(f"Token: {token}")

            print()

output_thread = threading.Thread(target=stream_tunnel_output, daemon=True)
output_thread.start()

try:
    return proc.wait()
except KeyboardInterrupt:
    proc.terminate()

    try:
        proc.wait(timeout=5)
    except subprocess.TimeoutExpired:
        proc.kill()

    return 130

def ngrok_command(argv: Sequence[str]) -> int:
    parser = argparse.ArgumentParser(prog="hao ngrok")

    parser.add_argument("--domain", required=True, help="Your fixed ngrok dev domain, like a...")

    parser.add_argument("--port", type=int, default=8765)

    parser.add_argument("--host", default="127.0.0.1")

    parser.add_argument("--token", default=None)

    parser.add_argument("--ngrok", default="ngrok")

    ns = parser.parse_args(argv)

    ngrok = shutil.which(ns.ngrok) if ns.ngrok == "ngrok" else ns.ngrok

    if not ngrok:
        print(
            "hao: ngrok is required for fixed free dev domains.\n"

            "Install it with: brew install ngrok/ngrok/ngrok\n"

            "Then run: ngrok config add-authtoken <YOUR_NGROK_AUTHTOKEN>",

            file=sys.stderr,
        )

    return 127

```

```

token = ns.token or os.environ.get("HAOLEME_TOKEN") or secrets.token_urlsafe(24)

local_url = f"http://{ns.host}:{ns.port}"

public_url = f"https://{remove_prefix(remove_prefix(ns.domain, 'https://'), 'http://').r...

print("Starting ??? local server...")

print(f"Local: {local_url}")

print(f"Server: {public_url}")

print(f"Token: {token}")

print("Starting ngrok fixed dev domain tunnel...")

server_thread = threading.Thread(

    target=serve,

    args=(ns.host, ns.port),

    kwargs={"token": token},

    daemon=True,

)

server_thread.start()

time.sleep(0.5)

proc = subprocess.Popen(

    [ngrok, "http", str(ns.port), "--url", remove_prefix(remove_prefix(ns.domain, "https...

    stdout=subprocess.PIPE,

    stderr=subprocess.STDOUT,

    text=True,

    bufsize=1,

)

print()

print("Use this in the Android app:")

print(f"Server: {public_url}")

print(f"Token: {token}")

print()

try:

    if proc.stdout is not None:

        for line in proc.stdout:

            sys.stdout.write(line)

            sys.stdout.flush()

        return proc.wait()

except KeyboardInterrupt:

    proc.terminate()

    try:

```

```

        proc.wait(timeout=5)

    except subprocess.TimeoutExpired:

        proc.kill()

    return 130

```

```

def project_command(argv: Sequence[str]) -> int:

```

```

    parser = argparse.ArgumentParser(prog="hao project")

    subparsers = parser.add_subparsers(dest="action")

    use_parser = subparsers.add_parser("use", help="Set the default project for future hao r...
    use_parser.add_argument("name", help="Project name, for example: website or paper-tests")

    subparsers.add_parser("clear", help="Stop assigning future runs to a default project.")
    subparsers.add_parser("status", help="Show the current default project.")

    ns = parser.parse_args(argv)

    if ns.action == "use":

        name = normalize_project_name(ns.name)

        if not name:

            print("hao project: project name cannot be empty", file=sys.stderr)

            return 2

        save_default_project(name)

        print(f"Default project: {name}")

        print("Future hao runs will be grouped there. Use --no-project for one run outside p...

        return 0

    if ns.action == "clear":

        save_default_project("")

        print("Default project cleared. Future hao runs will not be grouped by project.")

        return 0

    configured_project = configured_default_project()

    project = default_project()

    if project:

        source = "configured" if configured_project else "git"

        print(f"Default project: {project} ({source})")

        print("One-off override: hao --project other <command>")

        print("One-off outside projects: hao --no-project <command>")

    else:

        print("Default project: none")

        print("Set one with: hao project use NAME")

    return 0

```

```

def parse_run_args(argv: Sequence[str]) -> tuple[list[str], str | None]:

    args = list(argv)

    project_override: str | None = None

    command: list[str] = []

    i = 0

    while i < len(args):

        item = args[i]

        if item == "--":

            command = args[i + 1 :]

            break

        if item in {"--project", "-p"}:

            if i + 1 >= len(args):

                print("hao: --project requires a name", file=sys.stderr)

                return [], None

            project_override = normalize_project_name(args[i + 1])

            i += 2

            continue

        if item.startswith("--project="):

            project_override = normalize_project_name(item.split("=", 1)[1])

            i += 1

            continue

        if item == "--no-project":

            project_override = ""

            i += 1

            continue

        command = args[i:]

        break

    return command, project_override


def normalize_project_name(value: str | None) -> str:

    return (value or "").strip()[:80]


def configured_default_project() -> str:

    config_path = default_config_path()

    try:

        data = json.loads(config_path.read_text(encoding="utf-8"))

    except (OSError, json.JSONDecodeError):

        return ""

    if not isinstance(data, dict):

        return ""

```

```

projects = data.get("projects")

if not isinstance(projects, dict):
    return ""

return normalize_project_name(str(projects.get("default") or ""))


def default_project(cwd: str | Path | None = None) -> str:

    configured = configured_default_project()

    if configured:
        return configured

    return auto_git_project(cwd)


def auto_git_project(cwd: str | Path | None = None) -> str:

    workdir = str(Path(cwd or os.getcwd()).expanduser())

    try:
        proc = subprocess.run(
            ["git", "-C", workdir, "rev-parse", "--show-toplevel"],
            stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL,
            text=True,
            timeout=2,
            check=False,
        )

    except (OSError, subprocess.SubprocessError):
        return ""

    if proc.returncode != 0:
        return ""

    root = proc.stdout.strip()

    if not root:
        return ""

    return normalize_project_name(Path(root).name)


def save_default_project(project: str) -> None:

    config_path = default_config_path()

    config_path.parent.mkdir(parents=True, exist_ok=True)

    data: dict[str, object] = {}

    if config_path.exists():
        try:
            loaded = json.loads(config_path.read_text(encoding="utf-8"))

            if isinstance(loaded, dict):
                data = loaded

```

```

except (OSError, json.JSONDecodeError):

    data = {}

projects = data.get("projects")

if not isinstance(projects, dict):

    projects = {}

if project:

    projects["default"] = normalize_project_name(project)

else:

    projects.pop("default", None)

data["projects"] = projects

config_path.write_text(json.dumps(data, indent=2), encoding="utf-8")

try:

    config_path.chmod(0o600)

except OSError:

    pass

```

```

def status_command(argv: Sequence[str]) -> int:

    parser = argparse.ArgumentParser(prog="hao status")

    parser.add_argument("--limit", type=int, default=10)

    ns = parser.parse_args(argv)

    runs = RunStore().list_runs(limit=ns.limit)

    if not runs:

        print("No runs yet.")

        return 0

    for run in runs:

        exit_code = "" if run.exit_code is None else f" exit={run.exit_code}"

        project = f" [{run.project}]" if run.project else ""

        print(f"{run.id[:8]} {run.status:9} {run.started_at}{project} {run.commandText}{exit...")

    return 0

```

```

def cloud_login_command(argv: Sequence[str]) -> int:

    parser = argparse.ArgumentParser(prog="hao cloud-login")

    parser.add_argument("--api-url", default=os.environ.get("HAOLEME_CLOUD_URL", DEFAULT_CLO...

    parser.add_argument("--account", default=os.environ.get("HAOLEME_ACCOUNT", "default"))

    parser.add_argument("--token", default=os.environ.get("HAOLEME_ACCOUNT_TOKEN", ""))

    parser.add_argument("--skip-check", action="store_true")

    ns = parser.parse_args(argv)

    api_url = ns.api_url.strip().rstrip("/")

    if not api_url:

        print(

```

```

        "hao: missing --api-url.\n"

        "Example: hao cloud-login --api-url http://39.96.50.42 --account alice",

        file=sys.stderr,

    )

    return 2

token = ns.token.strip() or generate_account_token()

device_name = gethostname() or "??? CLI"

config = CloudConfig(

    api_url=api_url,

    account=ns.account.strip() or "default",

    token=token,

    device_id=generate_device_id(),

    device_name=device_name,

    machine_id=get_or_create_machine_id(),

)

if not ns.skip_check:

    try:

        CloudClient(config).health()

    except Exception as exc:

        print(f"hao: cloud health check failed: {exc}", file=sys.stderr)

        print("Use --skip-check if the cloud service is not deployed yet.", file=sys.std...

        return 1

config.save()

print("??? cloud login saved.")

print(f"Config: {default_config_path()}")

print(f"Server: {config.api_url}")

print(f"Account: {config.account}")

print(f"Device: {config.device_name}")

print(f"Token: {config.token}")

print()

print("Use the same Server and Token in the Android app.")

print("Future hao commands will sync to cloud automatically.")

_started, message = start_heartbeat_daemon()

print(f"Heartbeat: {message}")

return 0

```

```

def pairing_login_command(argv: Sequence[str]) -> int:

    parser = argparse.ArgumentParser(prog="hao login")

```

```

parser.add_argument("--api-url", default=os.environ.get("HAOLEME_CLOUD_URL", DEFAULT_CLO...
parser.add_argument("--device", default=os.environ.get("HAOLEME_DEVICE_NAME", gethostnam...
parser.add_argument("--timeout", type=int, default=300)

parser.add_argument("--new-device", action="store_true", help="Ignore the saved device i...
parser.add_argument("--reuse-saved-device", action="store_true", help="Trust and reuse t...
parser.add_argument("--yes", "-y", action="store_true", help="Re-login without prompting...

ns = parser.parse_args(argv)

api_url = ns.api_url.strip().rstrip("/")

client = PairingClient(api_url)

machine_id = get_or_create_machine_id()

existing_config = CloudConfig.load()

if existing_config is not None and not ns.yes and not confirm_relogin(existing_config):

    return 0

existing_device_id = reusable_login_device_id(existing_config, api_url, machine_id, ns.n...

public_key, private_key = generate_pair_keypair()

try:

    started = client.start(ns.device, existing_device_id, public_key)

except Exception as exc:

    print(f"hao: could not start login: {exc}", file=sys.stderr)

    print("If your ??? Cloud URL is different, use: hao login --api-url https://your-ser...

    return 1

code = str(started.get("code", ""))

pair_token = str(started.get("pairToken", ""))

if not code or not pair_token:

    print("hao: cloud did not return a pair code", file=sys.stderr)

    return 1

print("??? login")

print()

pair_url = build_pair_url(api_url, code)

print("Open the ??? Android app and enter this pair code:")

print()

print(f" {code}")

print()

print("Or scan this QR code with the phone camera:")

print()

print_qr(pair_url)

print()

print(f"Device: {ns.device}")

```

```

if existing_device_id:

    print("Reusing this computer's previous ??? device.")

elif ns.new_device:

    print("Pairing as a new ??? device.")

print("Waiting for pairing...")

paired = False

try:

    deadline = time.monotonic() + max(30, ns.timeout)

    while time.monotonic() < deadline:

        time.sleep(2)

        try:

            status = client.status(code, pair_token)

        except Exception:

            continue

        if status.get("status") != "confirmed":

            continue

        token = str(status.get("token", "")).strip()

        account = str(status.get("account", "default")).strip() or "default"

        device_id = str(status.get("deviceId", "")).strip()

        device_name = str(status.get("deviceName", ns.device)).strip() or ns.device

        encrypted_account_key = str(status.get("encryptedAccountKey", "")).strip()

        encryption_key = ""

        if encrypted_account_key:

            try:

                encryption_key = decrypt_account_key(encrypted_account_key, private_key)

            except Exception as exc:

                print(f"hao: could not decrypt ??? encryption key: {exc}", file=sys.stderr)

                return 1

        if not token:

            print("hao: cloud returned an empty account token", file=sys.stderr)

            return 1

    config = CloudConfig(

        api_url=api_url,

        account=account,

        token=token,

        device_id=device_id,

        device_name=device_name,

        machine_id=machine_id,

        encryption_key=encryption_key,

```

```

    )

    config.save()

    paired = True

    print()

    print("Login success.")

    print(f"Config: {default_config_path()}")

    print(f"Device: {device_name}")

    print("Encryption: enabled" if encryption_key else "Encryption: not enabled for ...")

    print("Future hao commands will sync to ??? Cloud automatically.")

    _started, message = start_heartbeat_daemon()

    print(f"Heartbeat: {message}")

    return 0

except KeyboardInterrupt:

    print("\nhao: login cancelled.", file=sys.stderr)

    return 130

finally:

    if not paired:

        try:

            client.cancel(code, pair_token)

        except Exception:

            pass

    print("hao: login timed out. Pair code cancelled. Run hao login again to get a new code....")

    return 1

def reusable_login_device_id(

    existing_config: CloudConfig | None,

    api_url: str,

    machine_id: str,

    force_new: bool = False,

    force_reuse: bool = False,

) -> str:

    if force_new or existing_config is None:

        return ""

    if existing_config.api_url.rstrip("/") != api_url.rstrip("/"):

        return ""

    if force_reuse:

        return existing_config.device_id

    if existing_config.machine_id and existing_config.machine_id == machine_id:

        return existing_config.device_id

```

```

return ""

def confirm_relogin(existing_config: CloudConfig) -> bool:

    print("??? is already logged in.")

    print("Server: hidden")

    print(f"Account: {existing_config.account or 'default'}")

    if existing_config.device_name or existing_config.device_id:

        print(f"Device: {existing_config.device_name or existing_config.device_id}")

    try:

        answer = input("Press Enter to re-login, or type n to cancel: ")

    except EOFError:

        print("hao: login cancelled. Use hao login --yes to re-login without a prompt.", fil...)

        return False

    return should_continue_relogin(answer)

def should_continue_relogin(answer: str) -> bool:

    value = answer.strip().lower()

    return value not in {"n", "no", "q", "quit", "cancel"}

def heartbeat_command(argv: Sequence[str]) -> int:

    parser = argparse.ArgumentParser(prog="hao heartbeat")

    parser.add_argument("action", nargs="?", choices=["start", "stop", "status", "run"], def...)

    ns = parser.parse_args(argv)

    if ns.action == "start":

        started, message = start_heartbeat_daemon()

        print(f"Heartbeat: {message}")

        return 0 if started else 1

    if ns.action == "stop":

        stopped, message = stop_heartbeat_daemon()

        print(f"Heartbeat: {message}")

        return 0 if stopped else 1

    if ns.action == "run":

        return heartbeat_run_foreground()

pid = read_heartbeat_pid()

if pid and is_process_running(pid):

    print(f"Heartbeat: running (pid {pid})")

    print(f"Interval: {HEARTBEAT_INTERVAL_SECONDS}s")

    print(f"Log: {heartbeat_log_path()}")

    state = read_heartbeat_state()

```

```

    if state:

        print(f"Last OK:    {state.get('lastOkAt') or 'never'}")

        if state.get("lastSyncAt"):

            print(f"Last sync: {state.get('lastSyncAt')} ({state.get('lastSyncedRuns', 0)...

        if state.get("lastError"):

            print(f"Last err:  {state.get('lastError')}")

    return 0

print("Heartbeat: stopped")

state = read_heartbeat_state()

if state:

    print(f"Last OK:    {state.get('lastOkAt') or 'never'}")

    if state.get("lastError"):

        print(f"Last err:  {state.get('lastError')}")

print("Run: hao heartbeat start")

return 1


def heartbeat_pid_path() -> Path:

    return default_config_path().with_name("heartbeat.pid")


def heartbeat_log_path() -> Path:

    return default_config_path().with_name("heartbeat.log")


def heartbeat_state_path() -> Path:

    return default_config_path().with_name("heartbeat.json")


def read_heartbeat_pid() -> int | None:

    try:

        raw = heartbeat_pid_path().read_text(encoding="utf-8").strip()

        return int(raw) if raw else None

    except (OSError, ValueError):

        return None


def is_process_running(pid: int) -> bool:

    if pid <= 0:

        return False

    try:

        os.kill(pid, 0)

    except ProcessLookupError:

        return False

    except PermissionError:

```

```

        return True

return True

def read_heartbeat_state() -> dict[str, object]:
    try:
        raw = heartbeat_state_path().read_text(encoding="utf-8")

        data = json.loads(raw)

        return data if isinstance(data, dict) else {}
    except (OSError, json.JSONDecodeError):
        return {}

def write_heartbeat_state(**fields: object) -> None:
    state = read_heartbeat_state()

    state.update(fields)

    state["updatedAt"] = time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime())

    path = heartbeat_state_path()

    path.parent.mkdir(parents=True, exist_ok=True)

    path.write_text(json.dumps(state, ensure_ascii=False, indent=2), encoding="utf-8")

    if os.name == "posix":
        try:
            path.chmod(0o600)

        except OSError:
            pass

def start_heartbeat_daemon() -> tuple[bool, str]:
    config = CloudConfig.load()

    if config is None:
        return False, "not configured (run hao login)"

    pid_path = heartbeat_pid_path()

    pid = read_heartbeat_pid()

    if pid and is_process_running(pid):
        state = read_heartbeat_state()

        if state.get("haolemeVersion") != __version__:
            stop_heartbeat_daemon()

        else:
            return True, f"already running (pid {pid})"

    pid_path.parent.mkdir(parents=True, exist_ok=True)

    unlink_missing(pid_path)

    log_path = heartbeat_log_path()

```

```

with log_path.open("a", encoding="utf-8") as log_file:

    proc = subprocess.Popen(

        [sys.executable, "-m", "haoleme", "heartbeat", "run"],

        stdin=subprocess.DEVNULL,

        stdout=log_file,

        stderr=subprocess.STDOUT,

        start_new_session=True,

        close_fds=True,

    )

    pid_path.write_text(f"{proc.pid}\n", encoding="utf-8")

    return True, f"started (pid {proc.pid})"

```

```

def stop_heartbeat_daemon() -> tuple[bool, str]:

    pid_path = heartbeat_pid_path()

    pid = read_heartbeat_pid()

    if not pid or not is_process_running(pid):

        unlink_missing(pid_path)

        return False, "not running"

    try:

        os.kill(pid, signal.SIGTERM)

    except OSError as exc:

        return False, f"could not stop pid {pid}: {exc}"

    unlink_missing(pid_path)

    return True, f"stopped (pid {pid})"

```

```

def heartbeat_initial_delay(config: CloudConfig) -> int:

    seed = config.device_id or config.machine_id or config.device_name or gethostname() or "...

    digest = hashlib.sha256(seed.encode("utf-8")).hexdigest()

    return int(digest[:8], 16) % HEARTBEAT_INTERVAL_SECONDS

```

```

def _parse_int(value: str):

    value = value.strip()

    try:

        return int(value)

    except (TypeError, ValueError):

        return None

```

```

def collect_gpu_stats() -> list:

    """Snapshot NVIDIA GPU utilization via nvidia-smi. Empty list if unavailable."""

    nvidia_smi = shutil.which("nvidia-smi")

```

```

if not nvidia_smi:

    return []

try:

    proc = subprocess.run(

        [

            nvidia_smi,

            "--query-gpu=index,name,utilization.gpu,memory.used,memory.total,temperature...",

            "--format=csv,noheader,nounits",

        ],

        stdout=subprocess.PIPE,

        stderr=subprocess.DEVNULL,

        timeout=5,

    )

except Exception:

    return []

if proc.returncode != 0:

    return []

gpus = []

for line in proc.stdout.decode("utf-8", errors="replace").strip().splitlines():

    parts = [p.strip() for p in line.split(",")]

    if len(parts) < 6:

        continue

    gpus.append({

        "index": _parse_int(parts[0]),

        "name": parts[1][:80],

        "utilization": _parse_int(parts[2]),

        "memoryUsed": _parse_int(parts[3]),

        "memoryTotal": _parse_int(parts[4]),

        "temperature": _parse_int(parts[5]),

    })

return gpus

```

```

def heartbeat_run_foreground() -> int:

    config = CloudConfig.load()

    if config is None:

        print("hao: cloud is not configured. Run hao login.", file=sys.stderr)

        return 1

    write_heartbeat_state(haolemeVersion=__version__)

    delay = heartbeat_initial_delay(config)

    print(f"??? heartbeat started. First heartbeat in {delay}s, then every {HEARTBEAT_INTERV...

```

```

try:
    time.sleep(delay)

while True:

    config = CloudConfig.load()

    if config is None:

        print("??? heartbeat paused: cloud config removed.", flush=True)

        time.sleep(HEARTBEAT_INTERVAL_SECONDS)

        continue

try:

    client = CloudClient(config, timeout=8.0)

    store = RunStore()

    interrupted = apply_cloud_interrupts(store, client)

    if interrupted:

        print(f"Applied {interrupted} cloud interrupt(s).", flush=True)

    recovered = reconcile_orphaned_running_runs(store, client)

    if recovered:

        print(f"Recovered {recovered} orphaned run(s).", flush=True)

    synced = sync_pending_runs(store, client, limit=100)

    if synced:

        print(f"Synced {synced} pending run(s).", flush=True)

        write_heartbeat_state(lastSyncAt=time.strftime("%Y-%m-%dT%H:%M:%SZ", tim...

    client.heartbeat(gpus=collect_gpu_stats())

    now_text = time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime())

    write_heartbeat_state(

        haolemeVersion=__version__,

        lastOkAt=now_text,

```

```

... [ MIDDLE OMITTED - ONLY FIRST 30 + LAST 30 PAGES ] ...
... [ MIDDLE OMITTED - ONLY FIRST 30 + LAST 30 PAGES ] ...

```

```

return value.startswith("dev_") and 8 <= len(value) <= 80 and all(ch.isalnum() or ch == ...

```

```

def first_query_value(query: dict[str, list[str]], name: str) -> str:

```

```

    values = query.get(name, [])

    return values[0].strip() if values else ""

```

```

def int_or_none(value: object) -> int | None:

```

```

    try:

        return int(value) if value is not None else None

    except (TypeError, ValueError):

        return None

```

```

def is_app_version_too_old(platform: str, app_version_code: int | None, min_android_version...

    return platform.lower() == "android" and app_version_code is not None and app_version_co...

def allow_rate(

    attempts_by_key: dict[str, list[float]],

    lock: threading.Lock,

    key: str,

    limit: int,

    window_seconds: int,

) -> bool:

    now = time.time()

    cutoff = now - window_seconds

    with lock:

        attempts = [value for value in attempts_by_key.get(key, []) if value >= cutoff]

        if len(attempts) >= limit:

            attempts_by_key[key] = attempts

            return False

        attempts.append(now)

        attempts_by_key[key] = attempts

        return True

def env_flag(name: str, default: bool = False) -> bool:

    value = os.environ.get(name, "")

    if not value:

        return default

    return value.strip().lower() not in {"0", "false", "no", "off"}

def server_requires_e2ee() -> bool:

    return env_flag("HA0LEME_REQUIRE_E2EE", env_flag("REMINDER_REQUIRE_E2EE", False))

def server_stats_token() -> str:

    return (os.environ.get("HA0LEME_STATS_TOKEN") or os.environ.get("REMINDER_STATS_TOKEN")) ...

def render_stats_html(stats: dict[str, Any]) -> str:

    def card(label: str, value: Any, sub: str = "") -> str:

        sub_html = f'<div class="s">{sub}</div>' if sub else ""

        return f'<div class="card"><div class="v">{value}</div><div class="l">{label}</div>{...

    cards = "".join([

        card("?? Online", stats["appOnline"], "? 5 ??"),

```

```

card("?? DAU", stats["appDau"], "? 24 ??"),

card("?? MAU", stats["appMau"], "? 30 ?"),

card("??? Accounts", stats["appTotalAccounts"], f'{stats["appInstalls"]} ???'),

card("???? Servers", f'{stats["serversOnline"]}/{stats["serversTotal"]}', "? 5 ??"),

])

return (

    "<!doctype html><html lang=\"zh\"><head>"

    "<meta charset=\"utf-8\">"

    "<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">"

    "<meta http-equiv=\"refresh\" content=\"30\">"

    "<title>??? ? ???</title><style>"

    ":root{color-scheme:light dark}"

    "body{font-family:-apple-system,system-ui,'PingFang SC',sans-serif;margin:0;padding:..."

    "h1{font-size:18px;font-weight:600;margin:4px 0 16px}"

    ".grid{display:grid;grid-template-columns:1fr 1fr;gap:12px}"

    ".card{background:#161c24;border:1px solid #232c38;border-radius:14px;padding:16px}"

    ".v{font-size:34px;font-weight:700;line-height:1.1}"

    ".l{font-size:13px;color:#9fb0c3;margin-top:6px}"

    ".s{font-size:11px;color:#67788c;margin-top:2px}"

    ".ts{font-size:11px;color:#67788c;margin-top:16px;text-align:center}"

    "</style></head><body>"

    "<h1>??? ? ???</h1>"

    f"<div class=\"grid\">{cards}</div>"

    f"<div class=\"ts\">??? {stats['generatedAt']} ? ? 30 ???</div>"

    "</body></html>"

)

```

```
def server_allows_legacy_admin_tokens() -> bool:
```

```
    return env_flag("HAOLEME_ALLOW_LEGACY_ADMIN_TOKENS", env_flag("REMINDER_ALLOW_LEGACY_ADM...
```

```
def server_allows_existing_legacy_accounts() -> bool:
```

```
    return env_flag("HAOLEME_ALLOW_EXISTING_LEGACY_ACCOUNTS", True)
```

```
def legacy_admin_token_allowed(db_path: Path, account_key: str) -> bool:
```

```
    if server_allows_legacy_admin_tokens():
```

```
        return True
```

```
    return server_allows_existing_legacy_accounts() and account_has_cloud_data(db_path, acco...
```

```
def iso_now() -> str:
```

```
    return datetime.now(timezone.utc).isoformat().replace("+00:00", "Z")
```

```

def iso_from_epoch(value: float) -> str:

    return datetime.fromtimestamp(float(value), timezone.utc).isoformat().replace("+00:00", ...)

def is_recent_timestamp(value: str, window_seconds: int) -> bool:

    return is_recent_timestamp_at(value, window_seconds, iso_now())

def is_recent_timestamp_at(value: str, window_seconds: int, now: str) -> bool:

    try:

        timestamp = datetime.fromisoformat(str(value).replace("Z", "+00:00")).timestamp()

        now_timestamp = datetime.fromisoformat(str(now).replace("Z", "+00:00")).timestamp()

    except (TypeError, ValueError):

        return False

    return (now_timestamp - timestamp) <= window_seconds

def serve(host: str, port: int, db_path: Path, min_android_version_code: int) -> None:

    server = HaolemeCloudServer((host, port), db_path, min_android_version_code)

    cloud_log({"ts": iso_now(), "event": "startup", "listen": f"http://{host}:{port}", "db":...

    server.serve_forever()

def main(argv: list[str] | None = None) -> int:

    args = list(sys.argv[1:] if argv is None else argv)

    if args and args[0] == "serve":

        args = args[1:]

    if args and args[0] == "backup":

        return backup_command(args[1:])

    if args and args[0] == "health":

        return health_command(args[1:])

    if args and args[0] == "audit-permissions":

        return audit_permissions_command(args[1:])

    if args and args[0] == "monitor":

        return monitor_command(args[1:])

    if args and args[0] == "stats":

        return stats_command(args[1:])

    parser = argparse.ArgumentParser(prog="haoleme-cloud")

    parser.add_argument("--host", default=os.environ.get("HAOLEME_CLOUD_HOST") or os.environ...)

    parser.add_argument("--port", type=int, default=int(os.environ.get("HAOLEME_CLOUD_PORT")...

    parser.add_argument("--db", default=os.environ.get("HAOLEME_CLOUD_DB") or os.environ.get...

    parser.add_argument(

```

```

        "--min-android-version-code",

        type=int,

        default=int(os.environ.get("MIN_ANDROID_VERSION_CODE", str(DEFAULT_MIN_ANDROID_VERSI...
    )

    ns = parser.parse_args(args)

    serve(ns.host, ns.port, Path(ns.db), ns.min_android_version_code)

    return 0

```

```
def backup_command(argv: list[str]) -> int:
```

```

    parser = argparse.ArgumentParser(prog="haoleme-cloud backup")

    parser.add_argument("--db", default=os.environ.get("HAOLEME_CLOUD_DB") or os.environ.get...

    parser.add_argument("--dir", default=os.environ.get("HAOLEME_CLOUD_BACKUP_DIR") or os.en...

    parser.add_argument("--keep", type=int, default=int(os.environ.get("HAOLEME_CLOUD_BACKUP...

    ns = parser.parse_args(argv)

    path = backup_database(Path(ns.db), Path(ns.dir), ns.keep)

    print(path)

    return 0

```

```
def health_command(argv: list[str]) -> int:
```

```

    parser = argparse.ArgumentParser(prog="haoleme-cloud health")

    parser.add_argument("--db", default=os.environ.get("HAOLEME_CLOUD_DB") or os.environ.get...

    parser.add_argument(

        "--min-android-version-code",

        type=int,

        default=int(os.environ.get("MIN_ANDROID_VERSION_CODE", str(DEFAULT_MIN_ANDROID_VERSI...
    )

    ns = parser.parse_args(argv)

    payload = health_payload(Path(ns.db), ns.min_android_version_code)

    print(json.dumps(payload, ensure_ascii=False, indent=2))

    return 0 if payload.get("ok") else 1

```

```
def audit_permissions_command(argv: list[str]) -> int:
```

```

    parser = argparse.ArgumentParser(prog="haoleme-cloud audit-permissions")

    parser.add_argument("--db", default=os.environ.get("HAOLEME_CLOUD_DB") or os.environ.get...

    ns = parser.parse_args(argv)

    payload = permission_audit(Path(ns.db))

    print(json.dumps(payload, ensure_ascii=False, indent=2))

    return 0 if payload.get("ok") else 1

```

```
def active_user_stats(db_path: Path) -> dict[str, Any]:
```

```
"""Active-user metrics for the operator: online / DAU / MAU.
```

```
A "user" is a distinct account_key. App activity is tracked via
app_tokens.last_used_at (touched on every app request, including the
background poll), so recency of that timestamp gives liveness.
"""
```

```
online_window = 300
```

```
day = 86400
```

```
month = 30 * day
```

```
with connect(db_path) as db:
```

```
    app_rows = db.execute(
```

```
        "SELECT account_key, last_used_at FROM app_tokens WHERE revoked_at = ''"
```

```
    ).fetchall()
```

```
    device_rows = db.execute(
```

```
        "SELECT account_key, last_seen_at FROM devices WHERE revoked_at = ''"
```

```
    ).fetchall()
```

```
def accounts_within(rows: list[sqlite3.Row], ts_key: str, seconds: int) -> set[str]:
```

```
    return {
```

```
        row["account_key"]
```

```
        for row in rows
```

```
        if is_recent_timestamp(row[ts_key] or "", seconds)
```

```
    }
```

```
return {
```

```
    "generatedAt": iso_now(),
```

```
    "appOnline": len(accounts_within(app_rows, "last_used_at", online_window)),
```

```
    "appDau": len(accounts_within(app_rows, "last_used_at", day)),
```

```
    "appMau": len(accounts_within(app_rows, "last_used_at", month)),
```

```
    "appTotalAccounts": len({row["account_key"] for row in app_rows}),
```

```
    "appInstalls": len(app_rows),
```

```
    "serversOnline": len(accounts_within(device_rows, "last_seen_at", online_window)),
```

```
    "serversTotal": len({row["account_key"] for row in device_rows}),
```

```
    "onlineWindowSeconds": online_window,
```

```
}
```

```
def stats_command(argv: list[str]) -> int:
```

```
    parser = argparse.ArgumentParser(prog="haoleme-cloud stats")
```

```
    parser.add_argument("--db", default=os.environ.get("HAOLEME_CLOUD_DB") or os.environ.get...
```

```
    parser.add_argument("--json", action="store_true", help="print raw JSON instead of a sum...
```

```
    ns = parser.parse_args(argv)
```

```
    stats = active_user_stats(Path(ns.db))
```

```
    if ns.json:
```

```
        print(json.dumps(stats, ensure_ascii=False, indent=2))
```

```

    return 0

print("??? cloud ? active users")

print(f"  Online (app, <=5min): {stats['appOnline']}")

print(f"  DAU    (app, <=24h): {stats['appDau']}")

print(f"  MAU    (app, <=30d): {stats['appMau']}")

print(f"  Total accounts:      {stats['appTotalAccounts']} ({stats['appInstalls']} app...)

print(f"  Servers online:      {stats['serversOnline']} / {stats['serversTotal']}")

print(f"  As of:                {stats['generatedAt']}")

return 0

```

```

def monitor_command(argv: list[str]) -> int:

    parser = argparse.ArgumentParser(prog="haoleme-cloud monitor")

    parser.add_argument("--db", default=os.environ.get("HAOLEME_CLOUD_DB") or os.environ.get...)
    parser.add_argument("--backup-dir", default=os.environ.get("HAOLEME_CLOUD_BACKUP_DIR") o...)
    parser.add_argument(
        "--min-android-version-code",
        type=int,
        default=int(os.environ.get("MIN_ANDROID_VERSION_CODE", str(DEFAULT_MIN_ANDROID_VERSI...
    )
    parser.add_argument(
        "--min-free-mb",
        type=int,
        default=int(os.environ.get("HAOLEME_MONITOR_MIN_FREE_MB", str(DEFAULT_MONITOR_MIN_FR...
    )
    parser.add_argument(
        "--max-backup-age-hours",
        type=int,
        default=int(os.environ.get("HAOLEME_MONITOR_MAX_BACKUP_AGE_HOURS", str(DEFAULT_MONIT...
    )
    parser.add_argument("--alert-webhook", default=os.environ.get("HAOLEME_ALERT_WEBHOOK_URL...

    ns = parser.parse_args(argv)

    payload = monitor_payload(
        Path(ns.db),
        Path(ns.backup_dir),
        ns.min_android_version_code,
        min_free_bytes=max(1, ns.min_free_mb) * 1024 * 1024,
        max_backup_age_hours=max(1, ns.max_backup_age_hours),
    )

    if not payload.get("ok") and ns.alert_webhook.strip():
        payload["alert"] = send_monitor_alert(payload, ns.alert_webhook)

```

```

    print(json.dumps(payload, ensure_ascii=False, indent=2))

    return 0 if payload.get("ok") else 1


if __name__ == "__main__":

    raise SystemExit(main())


===== FILE: crypto.py =====


from __future__ import annotations


import base64

import json

import os

from typing import Any


from cryptography.hazmat.primitives import hashes, serialization

from cryptography.hazmat.primitives.asymmetric import padding, rsa

from cryptography.hazmat.primitives.ciphers.aead import AESGCM


E2EE_VERSION = 1

RUN_E2EE_ALGORITHM = "AES-256-GCM"

KEY_WRAP_ALGORITHM = "RSA-OAEP-SHA256"

ENCRYPTED_FIELDS = ("command", "commandText", "cwd", "stdoutTail", "stderrTail", "outputTail")

METADATA_ENCRYPTED_FIELDS = ("command", "commandText", "cwd")

OUTPUT_ENCRYPTED_FIELDS = ("stdoutTail", "stderrTail", "outputTail")


def b64url_encode(value: bytes) -> str:

    return base64.urlsafe_b64encode(value).decode("ascii").rstrip("=")


def b64url_decode(value: str) -> bytes:

    padded = value.strip() + ("=" * (-len(value.strip()) % 4))

    return base64.urlsafe_b64decode(padded.encode("ascii"))


def generate_account_key() -> str:

    return b64url_encode(os.urandom(32))


def is_valid_account_key(value: str) -> bool:

    try:

        return len(b64url_decode(value)) == 32

    except Exception:

        return False

```

```

def generate_pair_keypair() -> tuple[str, bytes]:

    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

    public_key = private_key.public_key()

    public_pem = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo,
    ).decode("ascii")

    private_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption(),
    )

    return public_pem, private_pem


def decrypt_account_key(encrypted_key: str, private_key_pem: bytes) -> str:

    private_key = serialization.load_pem_private_key(private_key_pem, password=None)

    plaintext = private_key.decrypt(
        b64url_decode(encrypted_key),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None,
        ),
    )

    if len(plaintext) != 32:
        raise ValueError("invalid ??? encryption key")

    return b64url_encode(plaintext)


def encrypt_run_payload(run: dict[str, Any], account_key: str, *, include_output: bool = True...

    key = b64url_decode(account_key)

    if len(key) != 32:
        raise ValueError("??? encryption key must be 32 bytes")

    fields = ENCRYPTED_FIELDS if include_output else METADATA_ENCRYPTED_FIELDS

    cleartext = {field: run.get(field) for field in fields}

    cleartext_bytes = json.dumps(cleartext, ensure_ascii=False, separators=(",", ":")).encod...

    nonce = os.urandom(12)

    aad = str(run.get("id") or "").encode("utf-8")

    ciphertext = AESGCM(key).encrypt(nonce, cleartext_bytes, aad)

```

```

encrypted = dict(run)

encrypted["command"] = []

encrypted["commandText"] = "Encrypted command"

encrypted["cwd"] = ""

encrypted["stdoutTail"] = ""

encrypted["stderrTail"] = ""

encrypted["outputTail"] = ""

encrypted["e2ee"] = {

    "v": E2EE_VERSION,

    "alg": RUN_E2EE_ALGORITHM,

    "nonce": b64url_encode(nonce),

    "ciphertext": b64url_encode(ciphertext),

}

return encrypted

```

```

def encrypt_output_chunk(run_id: str, account_key: str, deltas: dict[str, str]) -> dict[str, ...

    key = b64url_decode(account_key)

    if len(key) != 32:

        raise ValueError("??? encryption key must be 32 bytes")

    cleartext = {field: str(deltas.get(field) or "") for field in OUTPUT_ENCRYPTED_FIELDS if...

    if not cleartext:

        raise ValueError("output chunk is empty")

    cleartext_bytes = json.dumps(cleartext, ensure_ascii=False, separators=(",", ":")).encod...

    nonce = os.urandom(12)

    aad = str(run_id or "").encode("utf-8")

    ciphertext = AESGCM(key).encrypt(nonce, cleartext_bytes, aad)

    return {

        "v": E2EE_VERSION,

        "alg": RUN_E2EE_ALGORITHM,

        "nonce": b64url_encode(nonce),

        "ciphertext": b64url_encode(ciphertext),

    }

```

```

def decrypt_output_chunk(run_id: str, account_key: str, chunk: dict[str, Any]) -> dict[str, ...

    key = b64url_decode(account_key)

    if int(chunk.get("v") or 0) != E2EE_VERSION:

        return {}

    nonce = b64url_decode(str(chunk.get("nonce") or ""))

    ciphertext = b64url_decode(str(chunk.get("ciphertext") or ""))

    aad = str(run_id or "").encode("utf-8")

```

```

cleartext = AESGCM(key).decrypt(nonce, ciphertext, aad)

fields = json.loads(cleartext.decode("utf-8"))

if not isinstance(fields, dict):

    return {}

return {field: str(fields.get(field) or "") for field in OUTPUT_ENCRYPTED_FIELDS if fiel...

```

```

def decrypt_run_payload(run: dict[str, Any], account_key: str) -> dict[str, Any]:

```

```

    e2ee = run.get("e2ee")

    if not isinstance(e2ee, dict):

        return run

    if int(e2ee.get("v") or 0) != E2EE_VERSION:

        return run

    key = b64url_decode(account_key)

    nonce = b64url_decode(str(e2ee.get("nonce") or ""))

    ciphertext = b64url_decode(str(e2ee.get("ciphertext") or ""))

    aad = str(run.get("id") or "").encode("utf-8")

    cleartext = AESGCM(key).decrypt(nonce, ciphertext, aad)

    fields = json.loads(cleartext.decode("utf-8"))

    if not isinstance(fields, dict):

        return run

    decrypted = dict(run)

    for field in ENCRYPTED_FIELDS:

        if field in fields:

            decrypted[field] = fields[field]

    output_chunks = run.get("outputChunks")

    if isinstance(output_chunks, list):

        merged_output = {field: "" for field in OUTPUT_ENCRYPTED_FIELDS}

        for chunk in output_chunks:

            if not isinstance(chunk, dict):

                continue

            piece = decrypt_output_chunk(str(run.get("id") or ""), account_key, chunk)

            for field, value in piece.items():

                merged_output[field] = merged_output.get(field, "") + value

        for field, value in merged_output.items():

            if value and not decrypted.get(field):

                decrypted[field] = value

            elif value:

                decrypted[field] = str(decrypted.get(field) or "") + value

    return decrypted

```

```
===== FILE: server.py =====
```

```
from __future__ import annotations
```

```
import json
```

```
import os
```

```
import secrets
```

```
from http import HTTPStatus
```

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
```

```
from urllib.parse import parse_qs, urlparse
```

```
from .compat import remove_prefix
```

```
from .store import RunStore
```

```
class HaolemeServer(ThreadingHTTPServer):
```

```
    def __init__(self, server_address: tuple[str, int], store: RunStore, token: str | None = ...
```

```
        super().__init__(server_address, HaolemeRequestHandler)
```

```
        self.store = store
```

```
        self.token = token
```

```
class HaolemeRequestHandler(BaseHTTPRequestHandler):
```

```
    server: HaolemeServer
```

```
    def do_GET(self) -> None:
```

```
        parsed = urlparse(self.path)
```

```
        if parsed.path == "/health":
```

```
            self.send_json({"ok": True})
```

```
            return
```

```
        if parsed.path.startswith("/api/") and not self.is_authorized():
```

```
            self.send_json({"error": "unauthorized"}, status=HTTPStatus.UNAUTHORIZED)
```

```
            return
```

```
        if parsed.path == "/api/runs":
```

```
            query = parse_qs(parsed.query)
```

```
            limit = parse_limit(query.get("limit", ["100"])[0])
```

```
            runs = [run.to_dict() for run in self.server.store.list_runs(limit=limit)]
```

```
            self.send_json({"runs": runs})
```

```
            return
```

```
        if parsed.path.startswith("/api/runs/"):

```

```
            run_id = remove_prefix(parsed.path, "/api/runs/")
```

```
            run = self.server.store.get_run(run_id)
```

```

        if run is None:

            self.send_json({"error": "run not found"}, status=HTTPStatus.NOT_FOUND)

            return

        self.send_json({"run": run.to_dict()})

        return

    if parsed.path == "/api/events":

        query = parse_qs(parsed.query)

        since = query.get("since", [None])[0]

        limit = parse_limit(query.get("limit", ["100"])[0])

        runs = [

            run.to_dict()

            for run in self.server.store.list_updated_since(since=since, limit=limit)

        ]

        latest = max((run["updatedAt"] for run in runs), default=since)

        self.send_json({"events": runs, "latest": latest})

        return

    self.send_json({"error": "not found"}, status=HTTPStatus.NOT_FOUND)

def do_DELETE(self) -> None:

    parsed = urlparse(self.path)

    if parsed.path.startswith("/api/") and not self.is_authorized():

        self.send_json({"error": "unauthorized"}, status=HTTPStatus.UNAUTHORIZED)

        return

    if parsed.path.startswith("/api/runs/"):

        run_id = remove_prefix(parsed.path, "/api/runs/")

        if self.server.store.delete_run(run_id):

            self.send_json({"deleted": True})

            return

        self.send_json({"error": "run not found"}, status=HTTPStatus.NOT_FOUND)

        return

    self.send_json({"error": "not found"}, status=HTTPStatus.NOT_FOUND)

def do_OPTIONS(self) -> None:

    self.send_response(HTTPStatus.NO_CONTENT)

    self.send_cors_headers()

    self.end_headers()

def send_json(self, payload: dict, status: HTTPStatus = HTTPStatus.OK) -> None:

    body = json.dumps(payload, ensure_ascii=False).encode("utf-8")

```

```

        self.send_response(status)

        self.send_header("Content-Type", "application/json; charset=utf-8")

        self.send_header("Content-Length", str(len(body)))

        self.send_cors_headers()

        self.end_headers()

        self.wfile.write(body)

def send_cors_headers(self) -> None:

    self.send_header("Access-Control-Allow-Origin", "*")

    self.send_header("Access-Control-Allow-Methods", "GET, DELETE, OPTIONS")

    self.send_header("Access-Control-Allow-Headers", "Authorization, Content-Type, X-Hao...

def is_authorized(self) -> bool:

    token = self.server.token

    if not token:

        return True

    auth = self.headers.get("Authorization", "")

    if auth.startswith("Bearer "):

        return secrets.compare_digest(remove_prefix(auth, "Bearer ").strip(), token)

    header_token = self.headers.get("X-Haoleme-Token", "")

    return bool(header_token) and secrets.compare_digest(header_token, token)

def log_message(self, fmt: str, *args: object) -> None:

    print(f"[hao server] {self.address_string()} - {fmt % args}")

def parse_limit(raw: str) -> int:

    try:

        return max(1, min(int(raw), 500))

    except ValueError:

        return 100

def serve(host: str, port: int, store: RunStore | None = None, token: str | None = None) -> ...

    actual_store = store or RunStore()

    actual_token = token or os.environ.get("HAOLEME_TOKEN")

    httpd = HaolemeServer((host, port), actual_store, actual_token)

    print(f"??? server listening on http://{host}:{port}")

    print(f"Database: {actual_store.db_path}")

    if actual_token:

        print("Auth: enabled")

    elif host in {"0.0.0.0", "::"}:

        print("WARNING: server is listening publicly without a token.")

```

```
httpd.serve_forever()
```

```
===== FILE: store.py =====
```

```
from __future__ import annotations
```

```
import json
```

```
import os
```

```
import sqlite3
```

```
from dataclasses import dataclass
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
from typing import Any
```

```
from ._compat import shlex_join
```

```
TERMINAL_STATUSES = {"succeeded", "failed", "cancelled"}
```

```
DEFAULT_OUTPUT_TAIL_CHARS = 300_000
```

```
MAX_OUTPUT_TAIL_CHARS = 1_000_000
```

```
class ClosingConnection(sqlite3.Connection):
```

```
    def __exit__(self, exc_type, exc_value, traceback) -> bool:
```

```
        try:
```

```
            return super().__exit__(exc_type, exc_value, traceback)
```

```
        finally:
```

```
            self.close()
```

```
def utc_now() -> str:
```

```
    return datetime.now(timezone.utc).isoformat(timespec="seconds")
```

```
def default_data_dir() -> Path:
```

```
    configured = os.environ.get("HAOLEME_HOME") or os.environ.get("REMINDER_HOME")
```

```
    if configured:
```

```
        return Path(configured).expanduser()
```

```
    home = Path.home()
```

```
    current = home / ".haoleme"
```

```
    legacy = home / ".haoleme"
```

```
    if legacy.exists() and not current.exists():
```

```
        return legacy
```

```
    return current
```

```

def default_db_path() -> Path:

    data_dir = default_data_dir()

    legacy_db = data_dir / "haoleme.db"

    if legacy_db.exists():

        return legacy_db

    return data_dir / "haoleme.db"


@dataclass(frozen=True)

class RunRecord:

    id: str

    command: list[str]

    cwd: str

    project: str

    status: str

    pid: int | None

    exit_code: int | None

    started_at: str

    ended_at: str | None

    updated_at: str

    stdout_tail: str

    stderr_tail: str

    output_tail: str

    cloud_synced_at: str


@property

def commandText(self) -> str:

    return shlex_join(self.command)


@classmethod

def from_row(cls, row: sqlite3.Row) -> "RunRecord":

    return cls(

        id=row["id"],

        command=json.loads(row["command"]),

        cwd=row["cwd"],

        project=row["project"] or "",

        status=row["status"],

        pid=row["pid"],

        exit_code=row["exit_code"],

        started_at=row["started_at"],

        ended_at=row["ended_at"],

        updated_at=row["updated_at"],

```

```

        stdout_tail=row["stdout_tail"] or "",
        stderr_tail=row["stderr_tail"] or "",
        output_tail=row["output_tail"] or "",
        cloud_synced_at=row["cloud_synced_at"] or "",
    )

```

```

def to_dict(self) -> dict[str, Any]:

```

```

    return {
        "id": self.id,
        "command": self.command,
        "commandText": self.commandText,
        "cwd": self.cwd,
        "project": self.project,
        "status": self.status,
        "pid": self.pid,
        "exitCode": self.exit_code,
        "startedAt": self.started_at,
        "endedAt": self.ended_at,
        "updatedAt": self.updated_at,
        "stdoutTail": self.stdout_tail,
        "stderrTail": self.stderr_tail,
        "outputTail": self.output_tail,
    }

```

```

class RunStore:

```

```

    def __init__(self, db_path: Path | str | None = None, output_tail_chars: int | None = No...

```

```

        self.db_path = Path(db_path) if db_path else default_db_path()

        self.output_tail_chars = normalize_output_tail_chars(output_tail_chars)

        self.db_path.parent.mkdir(parents=True, exist_ok=True)

        self.init_db()

```

```

    def connect(self) -> sqlite3.Connection:

```

```

        conn = sqlite3.connect(self.db_path, factory=ClosingConnection)

        conn.row_factory = sqlite3.Row

        return conn

```

```

    def init_db(self) -> None:

```

```

        with self.connect() as conn:

            conn.execute(

                """
                CREATE TABLE IF NOT EXISTS runs (
                    id TEXT PRIMARY KEY,
                    command TEXT NOT NULL,

```

```

        cwd TEXT NOT NULL,
        status TEXT NOT NULL,
        pid INTEGER,
        exit_code INTEGER,
        started_at TEXT NOT NULL,
        ended_at TEXT,
        updated_at TEXT NOT NULL,
        stdout_tail TEXT NOT NULL DEFAULT '',
        stderr_tail TEXT NOT NULL DEFAULT '',
        output_tail TEXT NOT NULL DEFAULT '',
        project TEXT NOT NULL DEFAULT '',
        cloud_synced_at TEXT NOT NULL DEFAULT ''
    )
    """

)

columns = {

    row["name"]

    for row in conn.execute("PRAGMA table_info(runs)").fetchall()
}

if "output_tail" not in columns:

    conn.execute("ALTER TABLE runs ADD COLUMN output_tail TEXT NOT NULL DEFAULT ...

if "project" not in columns:

    conn.execute("ALTER TABLE runs ADD COLUMN project TEXT NOT NULL DEFAULT ''")

if "cloud_synced_at" not in columns:

    conn.execute("ALTER TABLE runs ADD COLUMN cloud_synced_at TEXT NOT NULL DEFA...

    conn.execute(

        """
        UPDATE runs
        SET cloud_synced_at = updated_at
        WHERE cloud_synced_at = ''
        """

    )

conn.execute(

    "CREATE INDEX IF NOT EXISTS idx_runs_updated_at ON runs(updated_at DESC)"

)

conn.execute(

    "CREATE INDEX IF NOT EXISTS idx_runs_project_updated ON runs(project, update...

)

def create_run(self, run_id: str, command: list[str], cwd: str, project: str = "") -> None:

    now = utc_now()

    project_name = normalize_project_name(project)

    with self.connect() as conn:

        conn.execute(

            """
            INSERT INTO runs (
                id, command, cwd, project, status, pid, exit_code, started_at,
                ended_at, updated_at, stdout_tail, stderr_tail, output_tail, cloud_sync...
            ) VALUES (?, ?, ?, ?, 'created', NULL, NULL, ?, NULL, ?, '', '', '', '')
            """

            (run_id, json.dumps(command), cwd, project_name, now, now),

        )

```

```

def mark_running(self, run_id: str, pid: int) -> None:

    self._update(run_id, {"status": "running", "pid": pid})

def append_output(self, run_id: str, stream: str, text: str, max_chars: int | None = None):

    if stream not in {"stdout_tail", "stderr_tail"}:

        raise ValueError(f"unsupported stream: {stream}")

    limit = normalize_output_tail_chars(max_chars if max_chars is not None else self.out...

    with self.connect() as conn:

        conn.execute(

            f"""
            UPDATE runs
            SET {stream} = substr({stream} || ?, -?),
                output_tail = substr(output_tail || ?, -?),
                cloud_synced_at = '',
                updated_at = ?
            WHERE id = ?
            """,

            (text, limit, text, limit, utc_now(), run_id),

        )

def finish_run(self, run_id: str, exit_code: int) -> None:

    status = "succeeded" if exit_code == 0 else "failed"

    now = utc_now()

    with self.connect() as conn:

        conn.execute(

            """
            UPDATE runs
            SET status = ?, exit_code = ?, ended_at = ?, updated_at = ?, cloud_synced_at...
            WHERE id = ?
            """,

            (status, exit_code, now, now, run_id),

        )

def cancel_run(self, run_id: str, note: str = "") -> None:

    now = utc_now()

    limit = self.output_tail_chars

    with self.connect() as conn:

        if note:

            conn.execute(

                """
                UPDATE runs
                SET status = 'cancelled',
                    exit_code = -1,
                    ended_at = ?,
                    updated_at = ?,
                    stderr_tail = substr(stderr_tail || ?, -?),
                    output_tail = substr(output_tail || ?, -?),
                    cloud_synced_at = ''
                WHERE id = ?
                """,

                (now, now, note, limit, note, limit, run_id),
            )

```

```

    )

else:

    conn.execute(

        """
        UPDATE runs
        SET status = 'cancelled', exit_code = -1, ended_at = ?, updated_at = ?, ...
        WHERE id = ?
        """,

        (now, now, run_id),

    )

def get_run(self, run_id: str) -> RunRecord | None:

    with self.connect() as conn:

        row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()

        return RunRecord.from_row(row) if row else None

def delete_run(self, run_id: str) -> bool:

    with self.connect() as conn:

        cursor = conn.execute("DELETE FROM runs WHERE id = ?", (run_id,))

        return cursor.rowcount > 0

def list_runs(self, limit: int = 100) -> list[RunRecord]:

    with self.connect() as conn:

        rows = conn.execute(

            "SELECT * FROM runs ORDER BY started_at DESC LIMIT ?",

            (limit,),

        ).fetchall()

        return [RunRecord.from_row(row) for row in rows]

def list_active_runs(self, limit: int = 100) -> list[RunRecord]:

    with self.connect() as conn:

        rows = conn.execute(

            """
            SELECT * FROM runs
            WHERE status IN ('created', 'running')
            ORDER BY updated_at ASC
            LIMIT ?
            """,

            (limit,),

        ).fetchall()

        return [RunRecord.from_row(row) for row in rows]

def list_updated_since(self, since: str | None, limit: int = 100) -> list[RunRecord]:

    with self.connect() as conn:

        if since:

            rows = conn.execute(

                """
                SELECT * FROM runs

```

```

        WHERE updated_at > ?
        ORDER BY updated_at ASC
        LIMIT ?
        """
        (since, limit),
    ).fetchall()
else:
    rows = conn.execute(
        "SELECT * FROM runs ORDER BY updated_at ASC LIMIT ?",
        (limit,),
    ).fetchall()

    return [RunRecord.from_row(row) for row in rows]

def mark_cloud_synced(self, run_id: str) -> None:
    with self.connect() as conn:
        conn.execute(
            "UPDATE runs SET cloud_synced_at = ? WHERE id = ?",
            (utc_now(), run_id),
        )

def list_unsynced_runs(self, limit: int = 100) -> list[RunRecord]:
    with self.connect() as conn:
        rows = conn.execute(
            """
            SELECT * FROM runs
            WHERE cloud_synced_at = ''
            ORDER BY updated_at ASC
            LIMIT ?
            """
            (limit,),
        ).fetchall()

    return [RunRecord.from_row(row) for row in rows]

def count_unsynced_runs(self) -> int:
    with self.connect() as conn:
        row = conn.execute("SELECT COUNT(*) AS count FROM runs WHERE cloud_synced_at = '...'")
        return int(row["count"]) if row is not None else 0

def _update(self, run_id: str, fields: dict[str, Any]) -> None:
    if not fields:
        return

    fields["updated_at"] = utc_now()
    fields["cloud_synced_at"] = ""

    assignments = ", ".join(f"{name} = ?" for name in fields)

    values = list(fields.values()) + [run_id]

    with self.connect() as conn:

```

```
conn.execute(f"UPDATE runs SET {assignments} WHERE id = ?", values)
```

```
def normalize_project_name(value: str | None) -> str:
```

```
    return (value or "").strip()[:80]
```

```
def normalize_output_tail_chars(value: int | str | None = None) -> int:
```

```
    raw = value
```

```
    if raw is None:
```

```
        raw = os.environ.get("HAOLEME_CONSOLE_CHARS") or os.environ.get("HAOLEME_CONSOLE_CHA...
```

```
    try:
```

```
        parsed = int(raw)
```

```
    except (TypeError, ValueError):
```

```
        parsed = DEFAULT_OUTPUT_TAIL_CHARS
```

```
    return max(30_000, min(parsed, MAX_OUTPUT_TAIL_CHARS))
```